Name: Bidhan Pokhrel Gmail: bidhan.pokhrel.dubai@gmail.com M01051792

Task 1:

At first we need to generate the two sample tables: for that the following code was used. And the base line code was generated using genAI and modified as required.

bidhan-pokhrel Update generate_tables.py

Code Blame 44 lines (37 loc) · 1.52 KB

```python
import csv
import random
from datetime import datetime, timedelta

# Function to generate random dates
def random_date(start_date, end_date):
    time_between = end_date - start_date
    days = time_between.days
    random_days = random.randrange(days)
    return start_date + timedelta(days=random_days)

# Define date range for DOB (1985-01-01 to 2000-12-31)
start_date = datetime(1985, 1, 1)
end_date = datetime(2000, 12, 31)

# Sample names and course IDs
names = ["Alice Smith", "Bob Johnson", "Charlie Brown", "Diana Wilson", "Emma Davis",
         "Frank Miller", "Grace Lee", "Henry Clark", "Isabella Adams", "Jack Taylor"]
course_ids = ["CS101", "MATH201", "PHY301", "ENG401", "BIO501"]

# Generate Table A (StudentId, Name, DOB)
table_a = []
for i in range(1, 101):  # 100 students
    dob = random_date(start_date, end_date).strftime('%Y-%m-%d')
    table_a.append([i, random.choice(names), dob])

# Write Table A to CSV
with open('table_a.csv', 'w', newline='') as f:
    writer = csv.writer(f)
```

```
        writer.writerow(['StudentId', 'Name', 'DOB'])
        writer.writerows(table_a)

    # Generate Table B (StudentId, CourseId, Grade)
    table_b = []
    for i in range(1, 101):  # Each student has 1-3 courses
        num_courses = random.randint(1, 3)
        for _ in range(num_courses):
            table_b.append([i, random.choice(course_ids), round(random.uniform(60.0, 100.0), 1)])

    # Write Table B to CSV
    with open('table_b.csv', 'w', newline='') as f:
        writer = csv.writer(f)
        writer.writerow(['StudentId', 'CourseId', 'Grade'])
        writer.writerows(table_b)
```

Then, we need to code the mapper and reducer for the data set that we have generated using DeepSeek and we modified the code as required. (Though Gen AI gives the most of the code parts but it never gives the perfect codes so, we have to do some tweaks to make it work for us.)

Following are the codes of mapper.py for task 1:

CST4070 / mapper.py

bidhan-pokhrel Update mapper.py

Code Blame 39 lines (33 loc) · 1.34 KB

```
#!/usr/bin/env python3

import sys
from datetime import datetime

def mapper():
    for line in sys.stdin:
        line = line.strip()
        parts = line.split(',')

        if len(parts) == 3:
            # Attempt to parse as Table A (StudentId, Name, DOB)
            try:
                student_id = int(parts[0])
                name = parts[1]
                dob_str = parts[2]
                # Try to parse DOB to confirm it's Table A
                datetime.strptime(dob_str, '%Y-%m-%d')

                # Filter by DOB >= 1995-01-01
                if dob_str >= '1995-01-01':
                    print(f"{student_id}\tA\t{name}\t{dob_str}")
                continue # Processed as Table A, move to next line
            except ValueError:
                # If DOB parsing failed, it's likely not Table A format, try Table B
                pass # Continue to the next 'try' block

```

```
27
28              # Attempt to parse as Table B (StudentId, CourseId, Grade)
29              try:
30                  student_id = int(parts[0])
31                  course_id = parts[1]
32                  grade = float(parts[2]) # Grade should be a float
33                  print(f"{student_id}\tB\t{course_id}\t{grade}")
34              except ValueError:
35                  # Skip header or malformed lines that don't fit either pattern
36                  continue
37
38  if __name__ == "__main__":
39      mapper()
```

And the code for reducer.py for task 1 is:

CST4070 / reducer.py

bidhan-pokhrel Update reducer.py

Code Blame 45 lines (34 loc) · 1.44 KB

```
1   #!/usr/bin/env python3
2
3   import sys
4
5   def reducer():
6       current_student_id = None
7       student_info = {}
8       course_info = []
9
10        # Print headers first
11      # print("StudentId,Name,CourseId,Grade")
12
13      for line in sys.stdin:
14          line = line.strip()
15          parts = line.split('\t')
16
17          student_id = int(parts[0])
18          source_table = parts[1]
19          data = parts[2:]
20
21          if current_student_id and student_id != current_student_id:
22              # Process the previous student's data
23              if 'name' in student_info: # Ensure we have student details from Table A
24                  for course_data in course_info:
25                      print(f"{current_student_id},{student_info['name']},{course_data['course_id']},{course_data['grade']}")
26
```

```
                # Reset for the new student
                student_info = {}
                course_info = []

            current_student_id = student_id

            if source_table == 'A':
                student_info['name'] = data[0]
                student_info['dob'] = data[1] # Keep DOB for completeness, though filtered in mapper
            elif source_table == 'B':
                course_info.append({'course_id': data[0], 'grade': data[1]})

        # Process the last student's data
        if current_student_id and 'name' in student_info:
            for course_data in course_info:
                print(f"{current_student_id},{student_info['name']},{course_data['course_id']},{course_data['grade']}")

    if __name__ == "__main__":
        reducer()
```

Gen AI prompt and process:

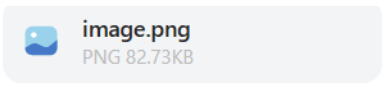
For generating two tables:

Step 1: I uploaded the sample image:

StudentId	Name	DOB
M575757	Alice	1994-01-05
M212121	Tom	1993-02-07
M989898	John	1995-06-02

StudentId	CourseId	Grade
M575757	CSD1414	pass
M575757	CSD5050	distinction
M575757	CSD5566	merit
M212121	CSD1414	distinction
M212121	CSD5050	distinction
M212121	CSD5566	distinction
M989898	CSD5050	merit
M989898	CSD5566	distinction

Step 2: I gave the prompt as: Write a Python script that generates two CSV files as below. The csv dataset should have 100 rows.



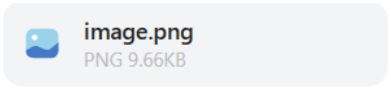
Write a Python script that generates two CSV files as below. The csv dataset should have 100 rows.

For generating mapper and reducer:

Step1: I uploaded the sql script:

```
select StudentId, Name, CourseId, Grade
from A
join B on A.StudentId = B.StudentId
where DOB >= 1995-01-01
```

Step 2: I gave prompt as : Generate mapper and reducer in python code that will run for two datasets that is generated by the above code and gives the exact same result as the sql query as provided. Generate two different files as mapper.py and reducer.py. The mapper.py and reducer.py should be able to run on Hadoop system as well as in local machine or cluster.



Generate mapper and reducer in python code that will run for two datasets that is generated by the above code and gives the exact same result as the sql query as provided. Generate two different files as mapper.py and reducer.py. The mapper.py and reducer.py should be able to run on Hadoop system as well as in local machine or cluster.

So, for now the files needed for task one is ready. We have files as: generate_tables.py, mapper.py and reducer.py

Now time to run.

To Run the machine locally we can use our machines that we make on clusters that will have Linux OS, or the same thing can be done using windows command prompt with slight modifications in code like: instead of “cat” we need to use “type” and few more modifications as shown below:

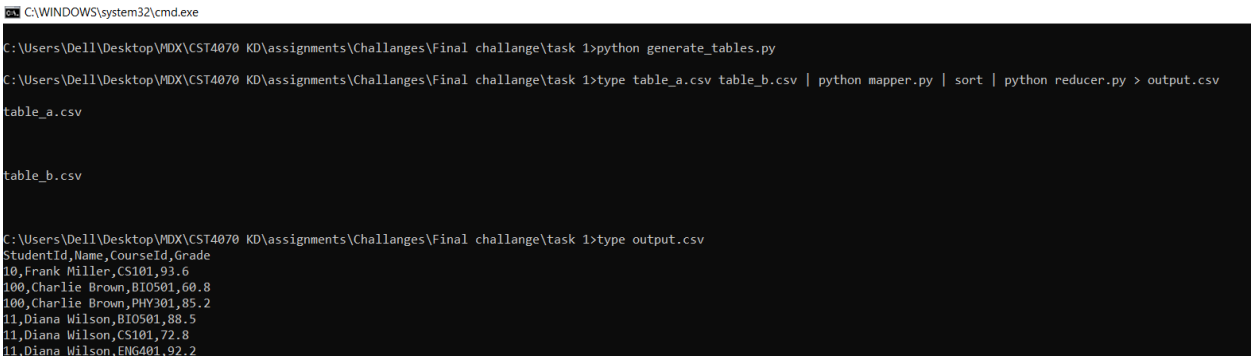


Image: Testing mapper and reducer in cmd on windows.

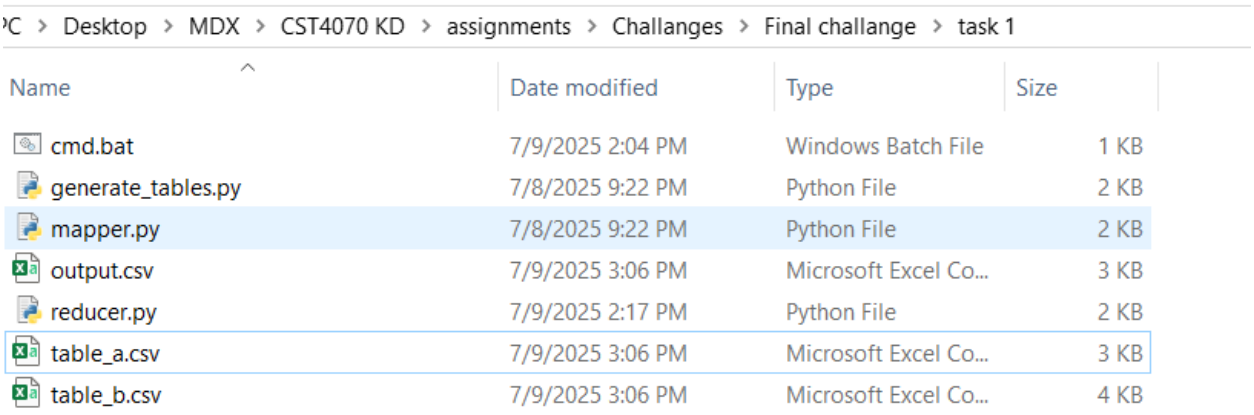


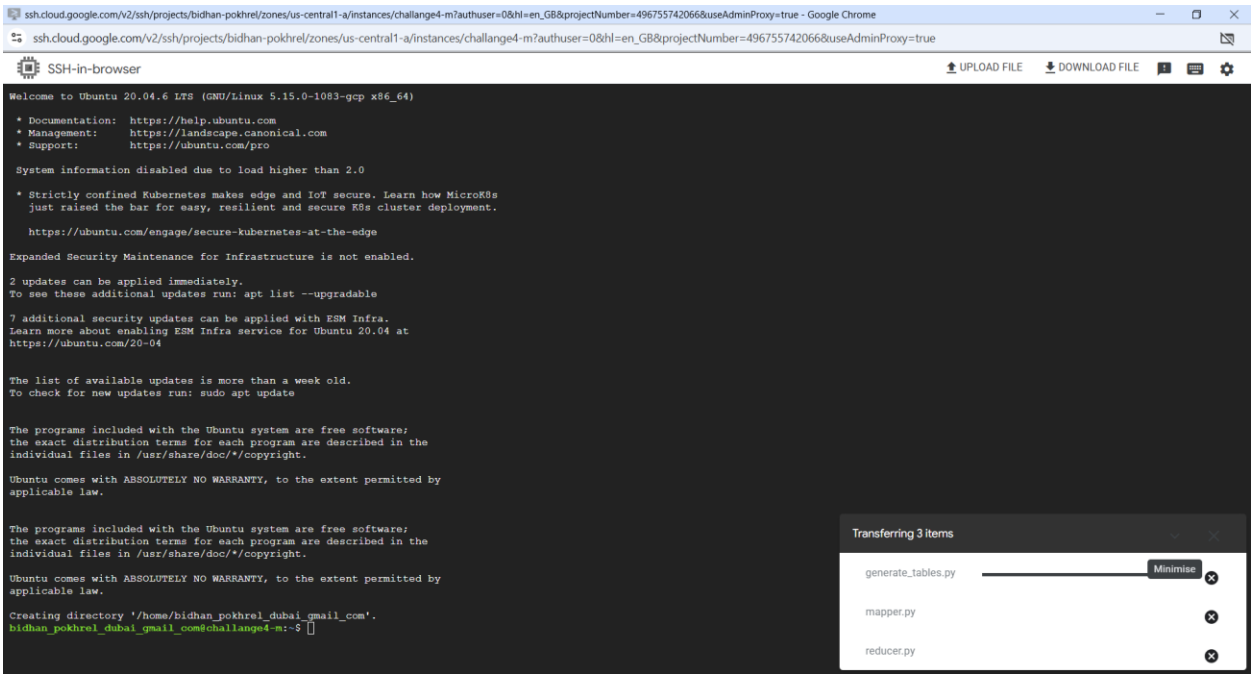
Image: Files that I have on my local windows machine.

```
C:\Users\Dell\Desktop\MDX\CST4070 KD\assignments\Challanges\Final challange\task 1>type output.csv
StudentId,Name,CourseId,Grade
10,Frank Miller,CS101,93.6
100,Charlie Brown,BIO501,60.8
100,Charlie Brown,PHY301,85.2
11,Diana Wilson,BIO501,88.5
11,Diana Wilson,CS101,72.8
11,Diana Wilson,ENG401,92.2
14,Grace Lee,ENG401,78.1
14,Grace Lee,PHY301,93.1
16,Diana Wilson,PHY301,61.2
2,Jack Taylor,BIO501,82.1
2,Jack Taylor,PHY301,75.1
2,Jack Taylor,PHY301,90.1
24,Alice Smith,CS101,67.8
24,Alice Smith,MATH201,63.0
24,Alice Smith,PHY301,92.7
25,Grace Lee,BIO501,66.1
25,Grace Lee,BIO501,67.1
25,Grace Lee,CS101,70.7
30,Frank Miller,BIO501,61.8
34,Alice Smith,BIO501,68.6
38,Emma Davis,MATH201,90.3
38,Emma Davis,PHY301,82.2
38,Emma Davis,PHY301,85.5
4,Charlie Brown,ENG401,91.1
4,Charlie Brown,MATH201,94.4
41,Henry Clark,CS101,98.6
41,Henry Clark,ENG401,87.8
43,Bob Johnson,MATH201,63.7
44,Jack Taylor,ENG401,94.2
```

Image showing final result

The same thing can be done using Dataproc cluster:

Following screen shots will show the detail steps:




```
Creating directory '/home/bidhan_pokhrel_dubai_gmail_com'.
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ la
.bash_logout  .bashrc  .cache  .profile  generate_tables.py  mapper.py  reducer.py
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ ls
generate_tables.py  mapper.py  reducer.py
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ chmod +x mapper.py
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ chmod +x reducer.py
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ cat table_a.csv table_b.csv |python mapper.py>mapout.csv
cat: table_a.csv: No such file or directory
cat: table_b.csv: No such file or directory
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ python generate_tables.py
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ ls
generate_tables.py  mapout.csv  mapper.py  reducer.py  table_a.csv  table_b.csv
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ cat table_a.csv table_b.csv |python mapper.py>mapout.csv
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ cat mapout.csv| sort -k1,1n |python reducer.py>redout.csv
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$
```

```
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ cat mapout.csv
1       A       Charlie Brown   1999-08-23
3       A       Jack Taylor     1995-01-17
5       A       Diana Wilson    2000-01-28
6       A       Isabella Adams  1997-07-06
9       A       Diana Wilson    1998-11-23
10      A       Charlie Brown   1995-06-04
```

```
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ cat redout.csv
StudentId,Name,CourseId,Grade
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
1,Charlie Brown,MATH201,67.1
3,Jack Taylor,CS101,90.4
```

Now time to run the same thing in Hadoop system:

#Making directory named as input inside Hadoop:

hadoop fs -mkdir -p /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input

#copy the files named table_a.csv and table_b.csv in Hadoop input folder:

hadoop fs -put table_a.csv table_b.csv /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input

#running mapreduce:

hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \

  -files mapper.py,reducer.py \

  -mapper mapper.py \

  -reducer reducer.py \

  -input /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input \

  -output /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output \

  -jobconf mapreduce.job.reduces=1

```
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
>    -files mapper.py,reducer.py \
>    -mapper mapper.py \
>    -reducer reducer.py \
>    -input /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input \
>    -output /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output \
>    -jobconf mapreduce.job.reduces=1
```

```
2025-07-09 09:40:36,570 INFO mapreduce.Job: Job job_1752053480600_0001 running in uber mode : false
2025-07-09 09:40:36,571 INFO mapreduce.Job:  map 0% reduce 0%
2025-07-09 09:40:44,668 INFO mapreduce.Job:  map 10% reduce 0%
2025-07-09 09:40:49,705 INFO mapreduce.Job:  map 40% reduce 0%
2025-07-09 09:40:55,741 INFO mapreduce.Job:  map 50% reduce 0%
2025-07-09 09:40:59,766 INFO mapreduce.Job:  map 70% reduce 0%
2025-07-09 09:41:00,771 INFO mapreduce.Job:  map 80% reduce 0%
2025-07-09 09:41:08,813 INFO mapreduce.Job:  map 100% reduce 0%
2025-07-09 09:41:15,856 INFO mapreduce.Job:  map 100% reduce 100%
2025-07-09 09:41:17,875 INFO mapreduce.Job: Job job_1752053480600_0001 completed successfully
2025-07-09 09:41:17,963 INFO mapreduce.Job: Counters: 55
        File System Counters
```

```
                Peak Reduce Virtual memory (bytes)=4731523072
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=16730
        File Output Format Counters
                Bytes Written=2371
2025-07-09 09:41:17,963 INFO streaming.StreamJob: Output directory: /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$
```

```
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ hadoop fs -ls /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/
Found 2 items
-rw-r--r--   2 bidhan_pokhrel_dubai_gmail_com hadoop          0 2025-07-09 09:41 /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/_SUCCESS
-rw-r--r--   2 bidhan_pokhrel_dubai_gmail_com hadoop       2371 2025-07-09 09:41 /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part-00000
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*
StudentId,Name,CourseId,Grade
1,Charlie Brown,MATH201,67.1
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
10,Charlie Brown,CS101,70.3
```

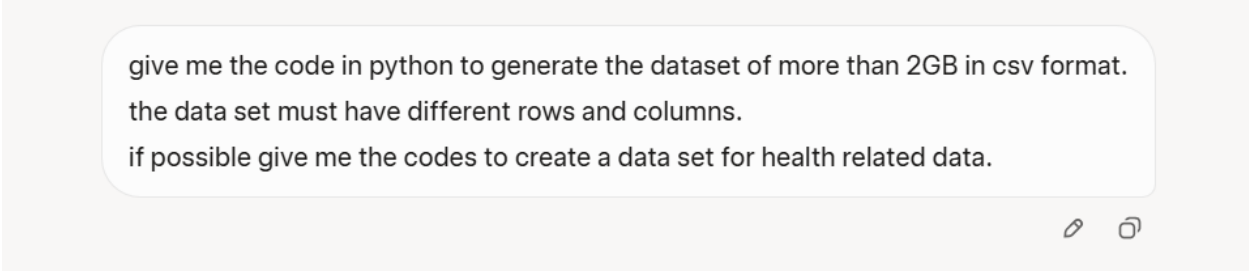
```
bidhan_pokhrel_dubai_gmail_com@challange4-m:~$ hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*
StudentId,Name,CourseId,Grade
1,Charlie Brown,MATH201,67.1
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
10,Charlie Brown,CS101,70.3
11,Charlie Brown,ENG401,88.3
14,Isabella Adams,PHY301,70.7
17,Charlie Brown,ENG401,99.3
17,Charlie Brown,BIO501,89.4
17,Charlie Brown,ENG401,83.7
18,Alice Smith,BIO501,76.6
18,Alice Smith,CS101,97.6
20,Bob Johnson,MATH201,76.7
20,Bob Johnson,ENG401,76.3
21,Jack Taylor,ENG401,96.1
21,Jack Taylor,ENG401,86.5
24,Charlie Brown,ENG401,73.0
24,Charlie Brown,BIO501,95.7
26,Emma Davis,CS101,79.0
26,Emma Davis,PHY301,85.9
29,Emma Davis,ENG401,76.4
29,Emma Davis,CS101,83.4
3,Jack Taylor,CS101,90.4
32,Charlie Brown,BIO501,69.9
35,Henry Clark,CS101,96.4
36,Isabella Adams,MATH201,77.6
36,Isabella Adams,CS101,70.4
37,Diana Wilson,BIO501,90.2
38,Emma Davis,CS101,74.4
42,Isabella Adams,PHY301,68.9
42,Isabella Adams,ENG401,92.9
48,Bob Johnson,BIO501,89.1
48,Bob Johnson,BIO501,95.5
5,Diana Wilson,BIO501,76.8
5,Diana Wilson,PHY301,81.3
52,Frank Miller,BIO501,93.4
52,Frank Miller,ENG401,86.9
52,Frank Miller,PHY301,71.3
55,Charlie Brown,MATH201,64.4
58,Bob Johnson,BIO501,95.4
6,Isabella Adams,BIO501,82.5
```

See output: hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*

Task 2:

For this task we need to generate the dataset first. To do so Gen AI was very useful.

The prompt that I gave to Gen AI : give me the code in python to generate the dataset of more than 2GB in csv format. the data set must have different rows and columns. if possible give me the codes to create a data set for health related data.



CST4070 / Data Gen / **Generate_largedata.py**

bidhan-pokhrel Rename Generate 2GB +data.py to Generate_largedata.py

Code | Blame | 76 lines (65 loc) · 2.69 KB

```python
import pandas as pd
import numpy as np
from faker import Faker
import random
from datetime import datetime, timedelta
import os

# Initialize Faker for realistic data
fake = Faker()

# Define parameters
columns = [
    'PatientID', 'FirstName', 'LastName', 'DOB', 'Gender',
    'BloodPressureSystolic', 'BloodPressureDiastolic', 'HeartRate',
    'CholesterolLevel', 'BloodSugarLevel', 'Diagnosis',
    'AdmissionDate', 'Weight', 'Height', 'BMI'
]

# Possible values for categorical fields
genders = ['Male', 'Female', 'Other']
diagnoses = ['Hypertension', 'Diabetes', 'Heart Disease', 'Asthma', 'None', 'COPD', 'Obesity']

# Function to generate a single row of health data
def generate_health_row():
    weight = random.uniform(50, 120)  # Weight in kg
    height = random.uniform(150, 200)  # Height in cm
```

```
        height = random.uniform(150, 200)  # Height in cm
        bmi = round(weight / ((height / 100) ** 2), 2)  # Calculate BMI
        dob = fake.date_of_birth(minimum_age=18, maximum_age=90)
        admission_date = fake.date_between(start_date='-2y', end_date='today')

        return {
            'PatientID': fake.uuid4(),
            'FirstName': fake.first_name(),
            'LastName': fake.last_name(),
            'DOB': dob.strftime('%Y-%m-%d'),
            'Gender': random.choice(genders),
            'BloodPressureSystolic': random.randint(90, 180),
            'BloodPressureDiastolic': random.randint(60, 120),
            'HeartRate': random.randint(60, 100),
            'CholesterolLevel': random.randint(120, 300),
            'BloodSugarLevel': random.randint(70, 200),
            'Diagnosis': random.choice(diagnoses),
            'AdmissionDate': admission_date.strftime('%Y-%m-%d'),
            'Weight': round(weight, 2),
            'Height': round(height, 2),
            'BMI': bmi
        }

    # Target file size: >2GB (approx 2,000,000,000 bytes)
    # Estimate row size: ~200-300 bytes per row (based on sample data)
    # Target ~10 million rows for ~2-3 GB
    num_rows_per_chunk = 100000  # Process in chunks to manage memory
    num_chunks = 250  # Adjust to achieve >2GB
    output_file = 'health_dataset.csv'
```

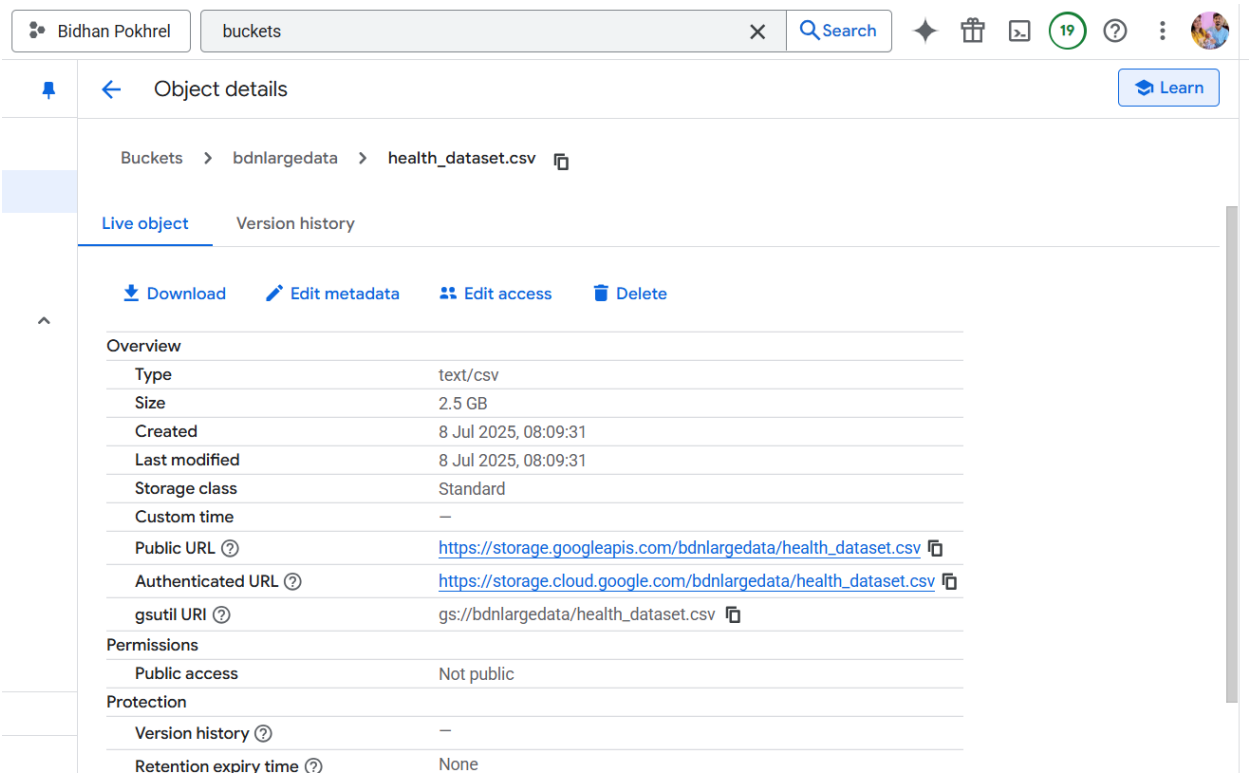
```python

    # Remove existing file if it exists
    if os.path.exists(output_file):
        os.remove(output_file)

    # Generate and write data in chunks
    for chunk in range(num_chunks):
        # Generate chunk of data
        data = [generate_health_row() for _ in range(num_rows_per_chunk)]
        df = pd.DataFrame(data, columns=columns)

        # Write to CSV (append mode after first chunk)
        mode = 'w' if chunk == 0 else 'a'
        header = True if chunk == 0 else False
        df.to_csv(output_file, mode=mode, header=header, index=False)

        print(f"Written chunk {chunk + 1}/{num_chunks}")

    print(f"Dataset generated at: {output_file}")
    # Check file size
    file_size = os.path.getsize(output_file) / (1024 ** 3)  # Size in GB
    print(f"Generated file size: {file_size:.2f} GB")
```

Here in this code we cn modify the line number 53, num_chunks, if we increase the number the dataset will be larger and it the chunk size is smaller then the dataset size will also be smaller. So, we can customize as per our needs.

```
C:\Users\Dell\Downloads>pip install faker
Collecting faker
  Using cached faker-37.4.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: tzdata in c:\users\dell\appdata\local\programs\python\python313\lib\site-packages (from faker) (2025.1)
Using cached faker-37.4.0-py3-none-any.whl (1.9 MB)
Installing collected packages: faker
Successfully installed faker-37.4.0

[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Dell\Downloads>
```

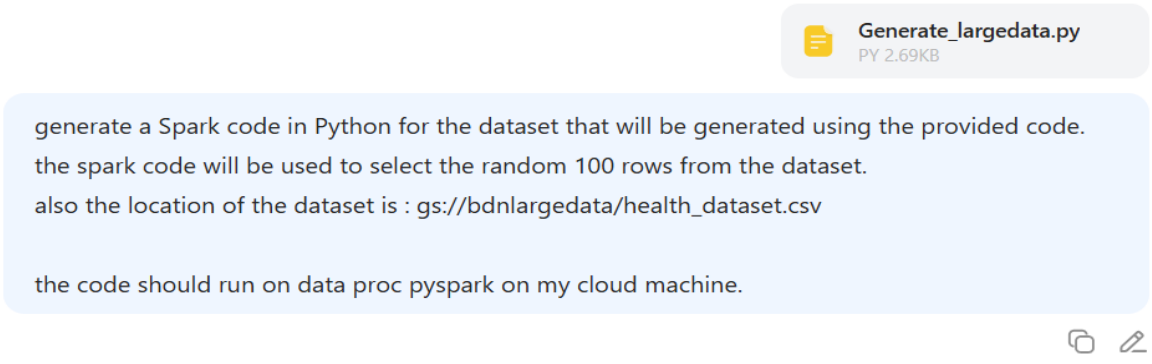
```
C:\Users\Dell\Downloads>python Generate_largedata.py
Written chunk 1/250
Written chunk 2/250
Written chunk 3/250
```

Then, I uploaded the dataset to my google bucket.

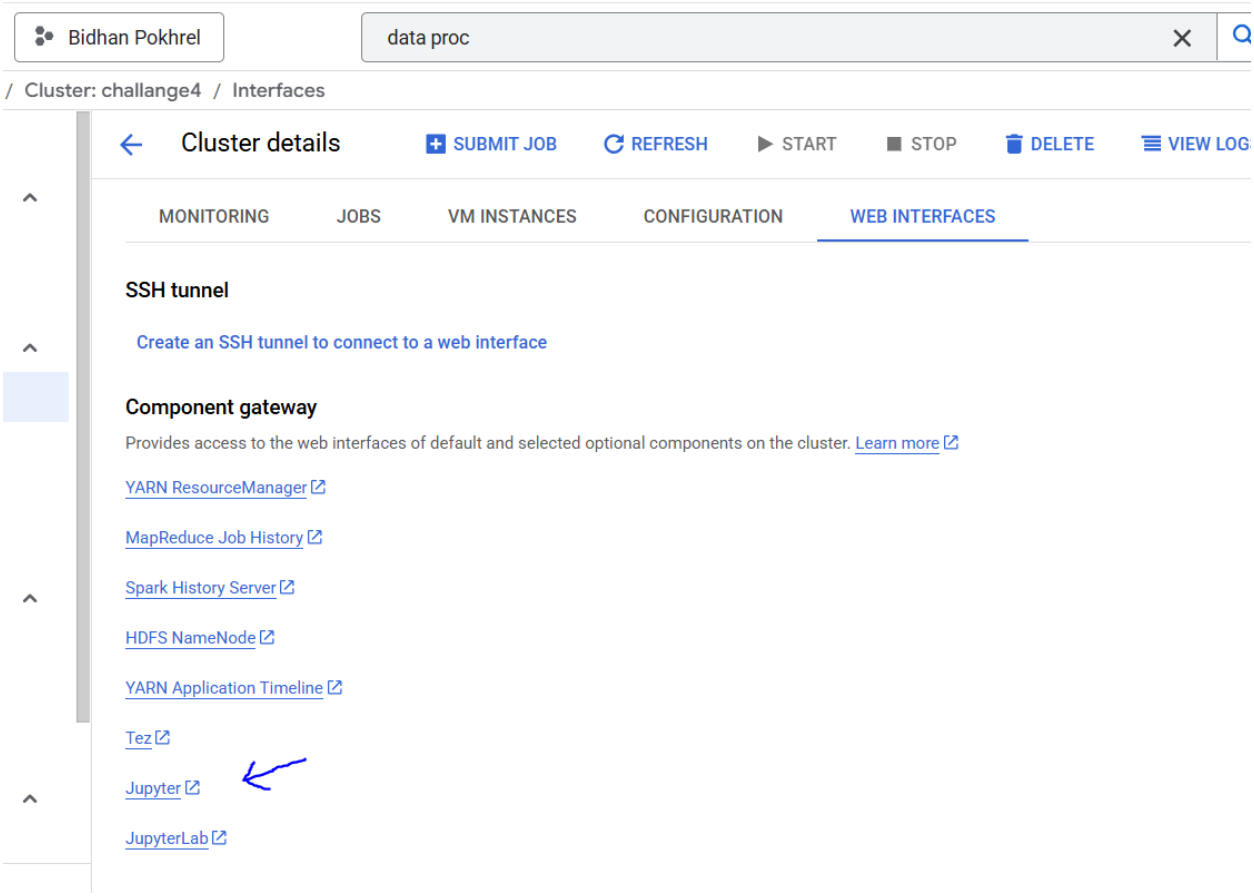


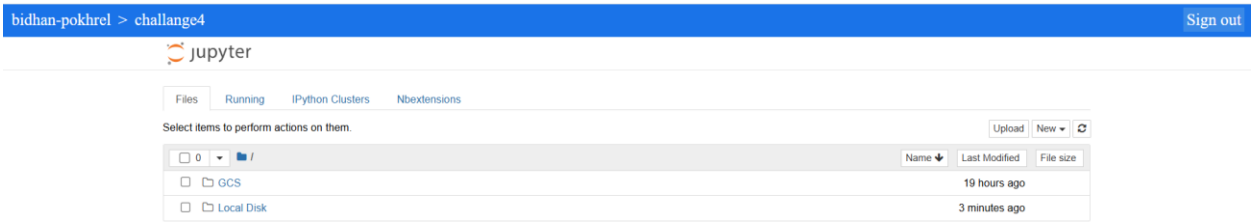
Now time to generate the Spark code.

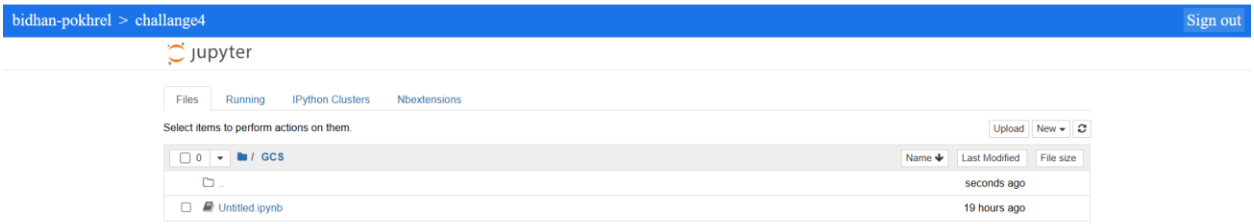
Gen AI prompt that I used to generate the Pyspark code:

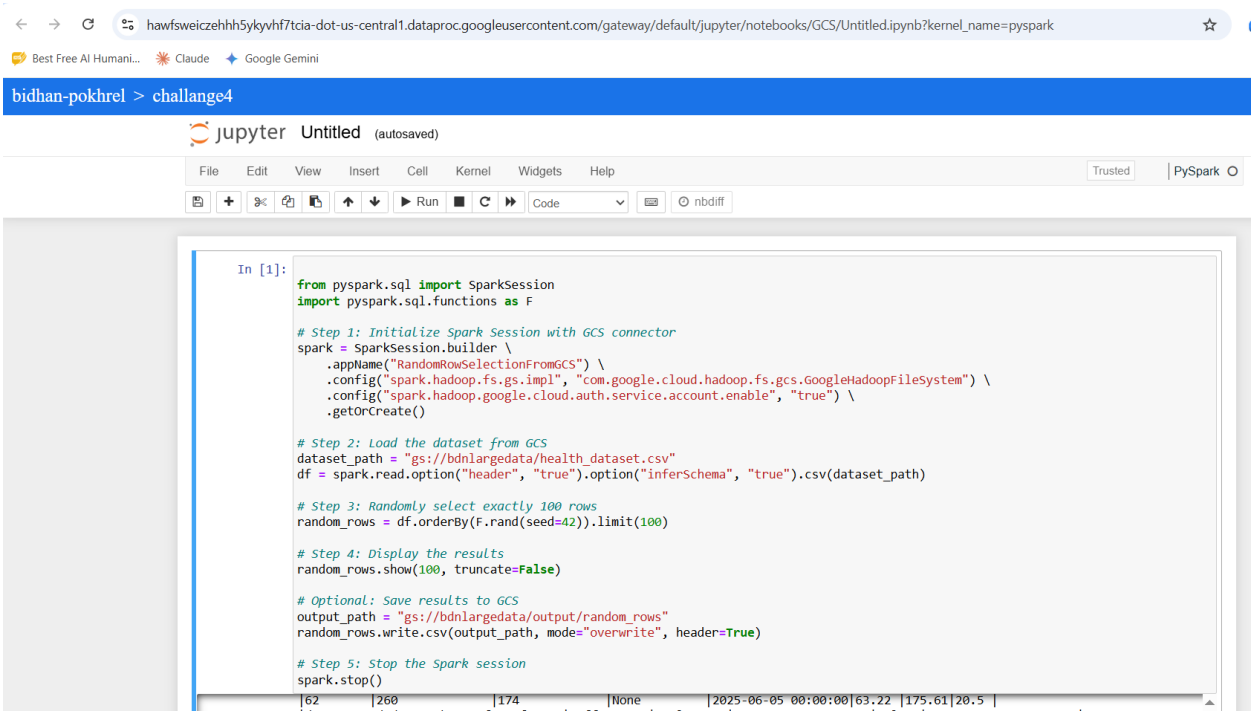


To run Spark on my cluster, I need to go to Web Interferences> Jupyter.

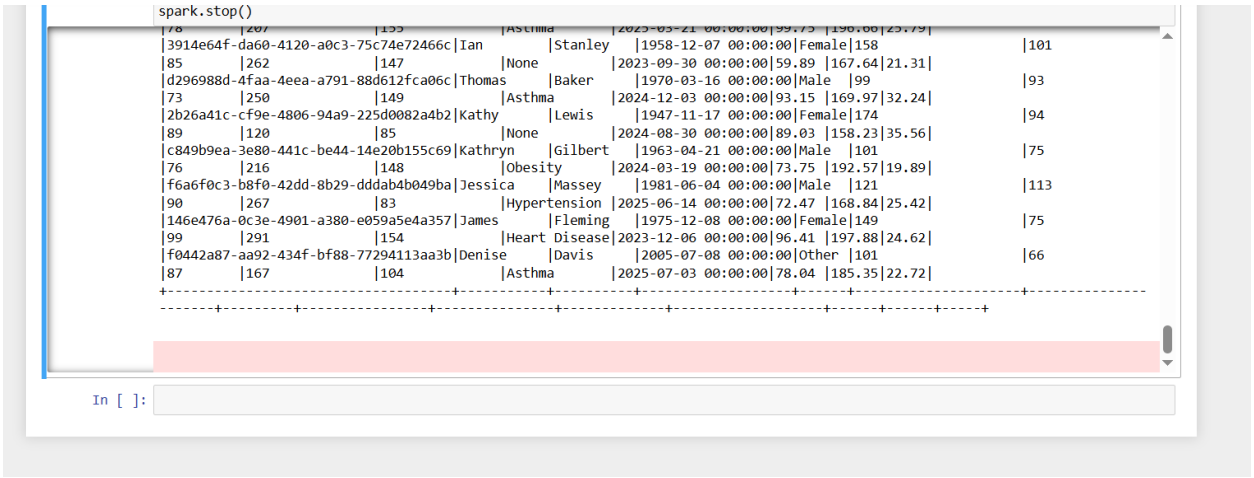








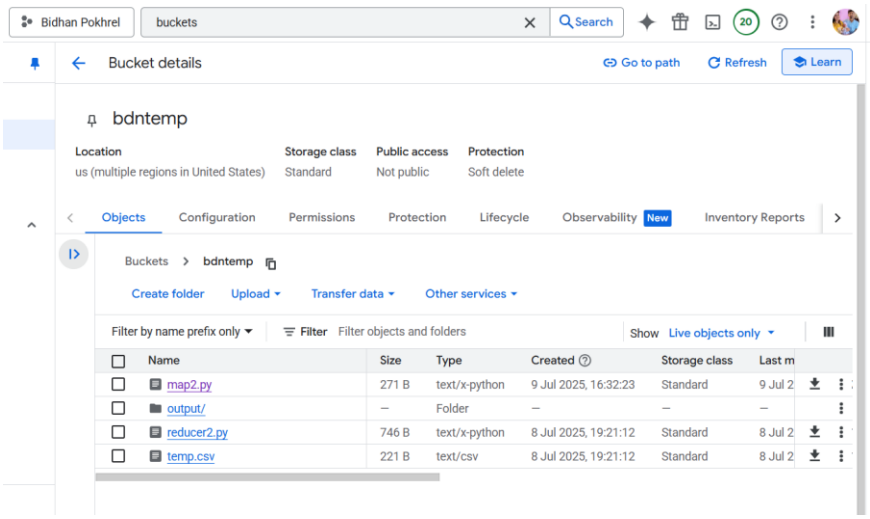

```
25/07/08 15:57:02 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

+------------------------------------+-----------+----------+-------------------+------+--------------------+--------------
-------+---------+---------------+--------------+-------------+-------------------+------+------+-----+
|PatientID                           |FirstName  |LastName  |DOB                |Gender|BloodPressureSystolic|BloodPressureDi
astolic|HeartRate|CholesterolLevel|BloodSugarLevel|Diagnosis    |AdmissionDate      |Weight|Height|BMI  |
+------------------------------------+-----------+----------+-------------------+------+--------------------+--------------
-------+---------+---------------+--------------+-------------+-------------------+------+------+-----+
|c14c0499-d44a-4d98-acd9-487378df9ab9|Caroline   |Rowe      |2005-08-30 00:00:00|Other |96                  |77
|97       |230             |87             |Asthma       |2024-12-24 00:00:00|101.35|190.17|28.02|
|474fd362-8a0a-4c09-a905-826dddb451b5|Michelle   |Hardin    |1944-07-21 00:00:00|Male  |170                 |83
|75       |177             |176            |None         |2025-06-13 00:00:00|85.36 |161.71|32.65|
|29d76c22-7aec-4b22-aef7-746ce8d406b3|Jeffrey    |Watson    |1946-08-04 00:00:00|Female|151                 |85
|80       |250             |127            |COPD         |2024-07-20 00:00:00|65.94 |199.6 |16.55|
|b4e25568-7543-4ce6-97bd-9435ededb870|Steven     |Harper    |1940-08-25 00:00:00|Female|150                 |78
|74       |255             |184            |Obesity      |2024-05-09 00:00:00|87.88 |160.07|34.3 |
|ba602e41-2ffa-4668-a1de-87297f449432|Brian      |Spencer   |1949-01-24 00:00:00|Male  |110                 |109
|65       |217             |112            |Heart Disease|2024-01-05 00:00:00|96.05 |162.28|36.47|
|e3bc011d-9cab-4ed9-a3e5-e9e8e2bb3837|Linda      |Mccoy     |1981-10-13 00:00:00|Female|143                 |100
```



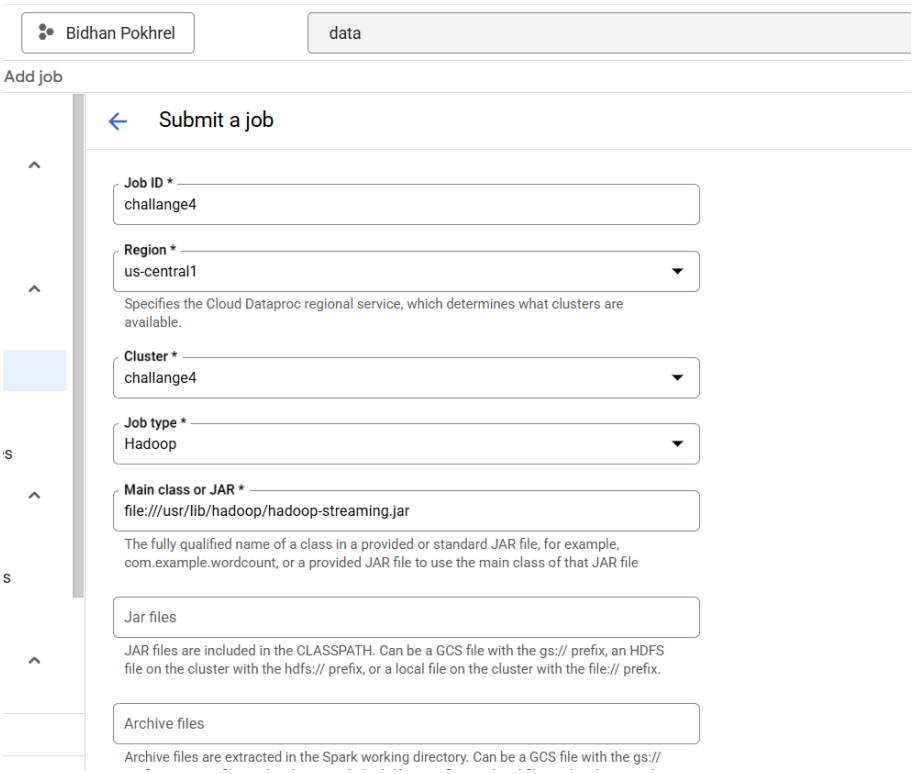
Task3:

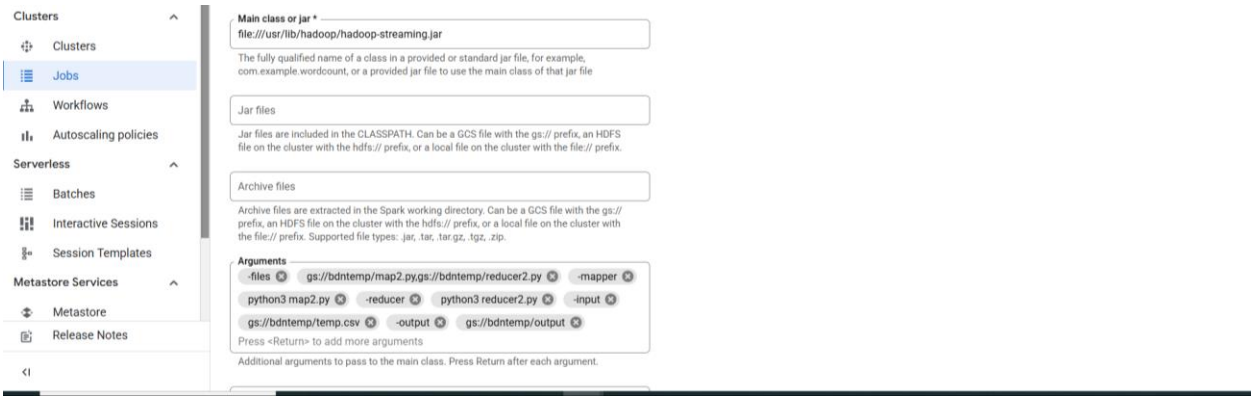
For task 3: I used the temp.csv for dataset and map2.py and reducer.py from our teams.

And I uploaded those three files to Google Bucket where the result will also be stored inside the output folder as specified in our arguments.

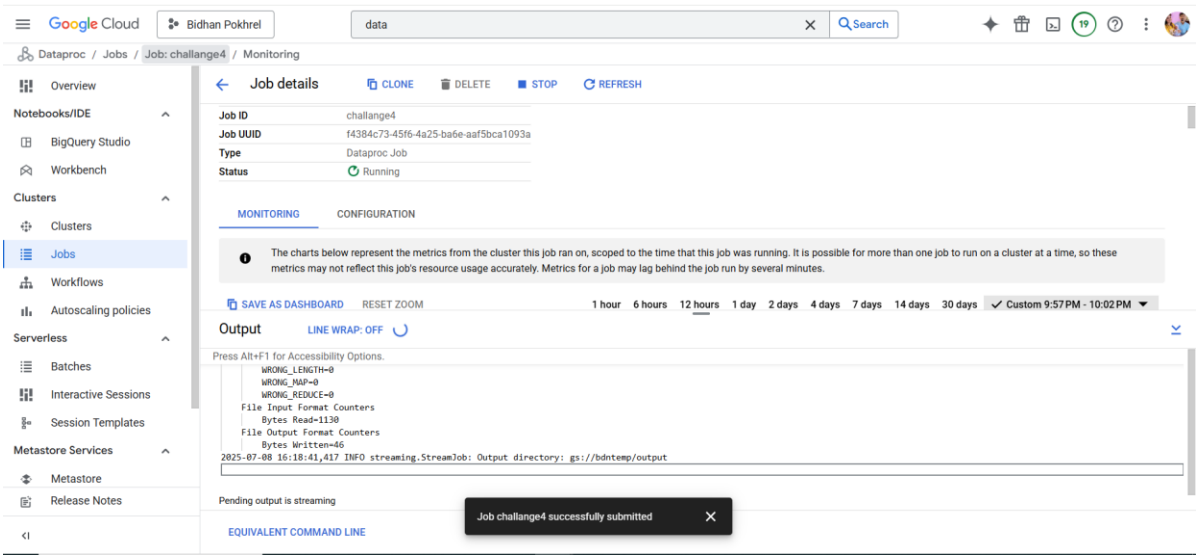


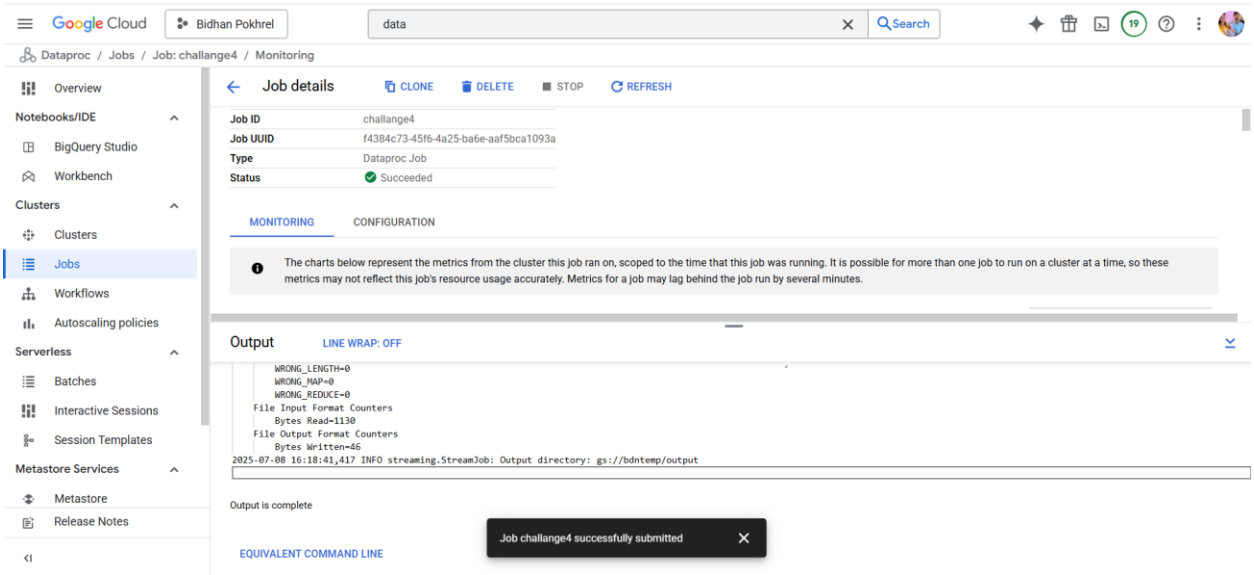
Job Creation:



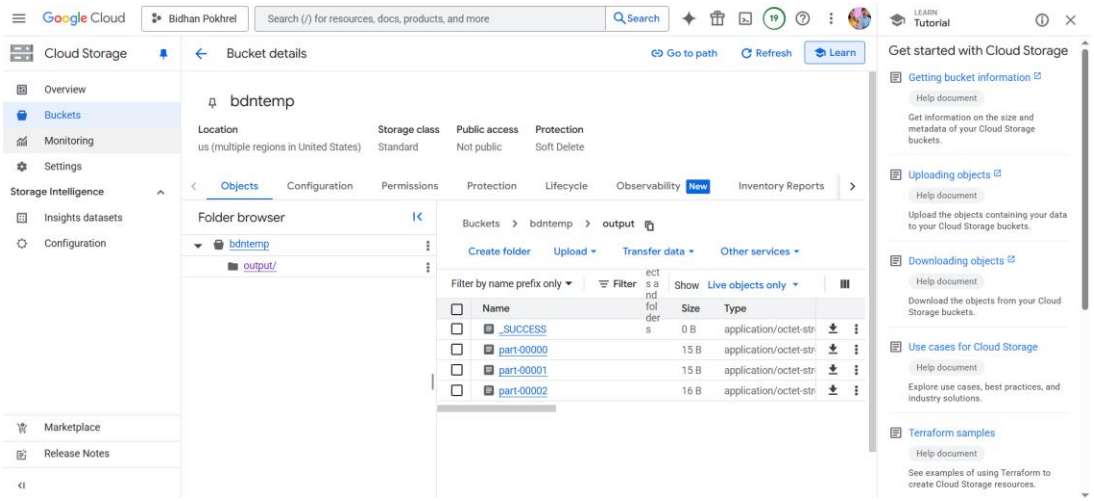


After this we can click submit and the task will execute the mapper and reducer for the given file on Hadoop system.

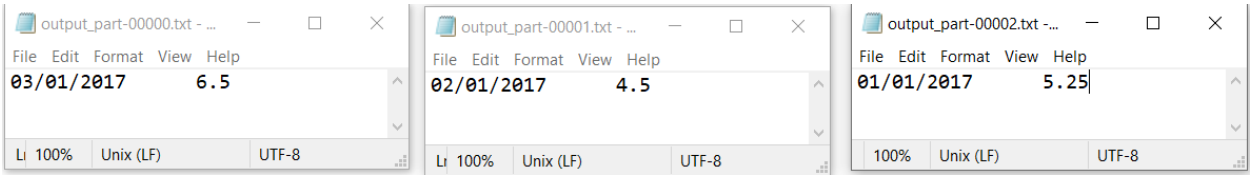




After successfully running the job output will be saved in the predefined output folder. Here, we have not defined the number of reducer so system automatically creates the reducers by default setting. Here, we can see the output are in three parts, names as: part-00000, part-00001,part-00002.



We can download and see the output as below:



This shows the daily average temperature from the input data.

If we had limited the reducer to one, then the result would be something like below:

01/01/2017 5.25

02/01/2017 4.5

03/01/2017 6.5